

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

KOMPILÁTOR DATALOGU S FUNKČNÝMI
SYMBOLMI DO RELAČNEJ ALGEBRY
BAKALÁRSKA PRÁCA

2026
MATÚŠ DUCHYŇA

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

KOMPILÁTOR DATALOGU S FUNKČNÝMI
SYMBOLMI DO RELAČNEJ ALGEBRY

BAKALÁRSKA PRÁCA

Študijný program: Informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra informatiky
Školiteľ: doc. Mgr. Tomáš Plachetka, Dr.

Bratislava, 2026
Matúš Duchyňa



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Matúš Duchyňa
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry
Compiler of Datalog with function symbols to relational algebra

Anotácia: Definovať rozšírenie Datalogu o funkčné symboly. Skonstruovať vhodnú formálnu gramatiku v syntaxi ANTLR. Implementovať kompilátor do relačnej algebry. Integrovať s kompilátorom relačnej algebry. Overiť funkčnosť na vhodných vstupoch.

Vedúci: doc. Mgr. Tomáš Plachetka, Dr.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Spôsob sprístupnenia elektronickej verzie práce:
archív

Dátum zadania: 21.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce

Čestné vyhlásenie: Čestne vyhlasujem, že celú bakalársku prácu na tému „Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry“, vrátane všetkých jej príloh a obrázkov, som vypracoval/vypracovala samostatne, a to s použitím literatúry uvedenej v priloženom zozname.

PodĎakovanie: TODO

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: datalog, relačná algebra, kompilátor, databázový systém

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords:

Obsah

Úvod	1
1 Datalog	3
1.1 Definície	3
1.2 Rozšírenie datalogu o funkčné symboly	6
1.3 Vstavane predikáty	7
1.4 Syntax rozšíreného datalogu	8
1.5 Sémantika datalogu	9
2 Relačná algebra	13
2.1 Definície	13
2.1.1 Učebnicová relačná algebra	13
2.1.2 Naša relačná algebra	17
2.2 Inklúzia medzi učebnicovou a našou relačnou algebrou	20
3 Kompilátor	23
3.1 ANTLR	23
3.1.1 Rýchlokurz formálnych jazykov	23
3.1.2 Syntaktická analýza	24
3.2 Gramatika datalogu pre ANTLR	28
3.3 Algoritmus prekladu	31
3.3.1 Generácia AST	32
3.3.2 Preklad do relačnej algebry	34
Záver	40
Literatúra	43

Úvod

Klasické moderné databázové systémy poskytujú používateľom na vytváranie dotazov zväčša deklaratívne jazyky, ako napríklad SQL. Výhodou deklaratívneho jazyka je to, že používateľ sa nemusí zaoberať samotným algoritmom, ktorý vypočíta výsledok dotazu. Súčasťou klasických databázových modelov je tzv. *optimalizér*. Tento komponent za pomoci rôznych metadát o databáze vytvorí zo SQL dotazu *vykonávací plán*, t.j. algoritmus výpočtu dotazu. Avšak optimalizér nie vždy vytvorí *optimálny* vykonávací plán. Prinútiť databázový systém používať konkrétny vykonávací plán nie je väčšinou jednoduché. Preto existujú experti, ktorí sa zaoberajú výlučne touto problematikou. Klasické postupy optimalizácie zahŕňajú používanie *hintov*, prepisovanie dotazu na iný ekvivalentný dotaz alebo explicitné nariadenie použitia konkrétneho plánu. Vykonávacie plány však väčšinou používajú veľké množstvo komplikovaných operácií s množstvom parametrov a bez kvalitnej dokumentácie. Kvôli tomu je písanie konkrétneho plánu nepraktické a neefektívne.

Náš databázový systém¹ používa ako dotazovací jazyk relačnú algebru. Konkrétne ide o relačnú algebru s piatimi operátormi a podporou funkčných symbolov. Viac o nej je uvedené v kapitole 2. Relačná algebra nám priamo umožňuje špecifikovať algoritmus materializácie dotazu. Pri jej návrhu sme sa zamerali na jednoduchosť, čo sa však odrazilo na veľkosti dotazov. *Loader*² z relačnej algebry do programovacieho jazyka Java a implementáciu jednotlivých funkcií v jazyku Java vytvorili kolegovia Biriukova [2] a Magát [3].

Relačná algebra je síce silným nástrojom na určenie presného algoritmu výpočtu dotazu ale pre bežného používateľa je príliš nízkoúrovňová. Tvorba zložitejších dotazov vyžaduje manuálne skladanie operátorov čo zvyšuje nečitateľnosť kódu a riziko chyby. Tu vzniká priestor pre datalog, deklaratívny dotazovací jazyk ktorý je založený na matematickej logike. Datalog umožňuje efektívne definovať zložité dotazy pomocou pravidiel, ktoré sú často omnoho stručnejšie ako ekvivalentné zápisy v SQL alebo relačnej algebre.

Cieľom mojej bakalárskej práce je pridať datalog s funkčnými symbolmi ako ďalší

¹Pod *naším* databázovým systémom myslím experimentálny systém, na ktorom pracuje alebo pracovali môj školiťel a kolegovia pod jeho vedením

²Kompilátor

dotazovací jazyk do nášho systému. Konkrétne ide o vytvorenie kompilátora z datalogu do relačnej algebry tak, aby bol použiteľný aj samostatne, bez väzby na konkrétny databázový systém. Dôležité bude spracovanie funkčných symbolov, ktoré sa v štandardnom datalogu (resp. prologu) nevyskytujú. Kompilátor musí byť schopný overiť syntaktickú a sémantickú správnosť datalogového programu a následne vygenerovať čo najoptimálnejší ekvivalentný kód v relačnej algebre.

Podobným softvérom je aj *Soufflé* [8]. *Soufflé* kompilátor taktiež vykonáva sémantickú kontrolu, ale následne preloží kód datalogu do tzv. *Relation Algebra Machine*. Z *RAM* sa potom vygeneruje vysoko optimalizovaný a paralelizovaný C++ kód. Naše riešenie sa síce nezameriava na rýchlosť preloženého dotazu, ale na jeho jednoduchosť. Náš datalog aj relačná algebra obsahujú iba tie najdôležitejšie komponenty, ale zároveň zanechávajú dotazovaciu silu.

Zásadným rozdielom oproti nášmu riešeniu je, že *Soufflé* nepodporuje funkčné symboly v podobe, ktorú by sme potrebovali. Rovnako *Soufflé* nepodporuje ani priame využívanie vstavaných funkcií databázy (*built-in functions*), čo obmedzuje jeho schopnosť spolupracovať s existujúcimi databázovými systémami. Náš prístup naopak počíta so vstavanou podporou funkcií priamo v relačnej algebre a databáze, čím umožňuje vykonávať komplexné transformácie dát už počas samotného procesu materializácie.

Okrem rozdielu v cieľovej platforme (C++ vs. Java) je dôležitá aj miera abstrakcie vygenerovanej relačnej algebry. Zatiaľ čo *RAM* je nízkoúrovňová inštrukčná sada navrhnutá pre konkrétny stroj, naša relačná algebra si zachováva matematickú podobu a je bližšia teoretickým definíciám. To umožňuje jednoduchšiu kontrolu správnosti prekladu a prípadnú manuálnu kontrolu vygenerovaného prekladu.

Práca je rozdelená na tri časti. V kapitole 1 bližšie popisujeme datalog ako jazyk logiky a definujeme jeho rozšírenie o funkčné symboly. Pri snahe osamostatniť náš kompilátor od nášho databázového systému sme identifikovali požiadavky na relačnú algebru. Všetky zmeny³ a ich odôvodnenia sú popísané v kapitole 2. V záverečnej kapitole 3 popisujeme samotný kompilátor, jeho jednotlivé časti vrátane gramatiky nášho datalogu a algoritmu prekladu.

³Oproti relačnej algebre definovanej v [2]

Kapitola 1

Datalog

1.1 Definície

V nasledujúcej kapitole formálne definujeme Datalog vo forme, v akej s ním budeme vo zvyšku práce pracovať.

Poznámka 1.1.1. $\top$ označuje pravdu (hodnotu *true*). $\perp$ označuje nepravdu (hodnotu *false*). $\mathbb{U}$ označuje hodnotu *unknown*.

Definícia 1.1.1. *n*-árna relácia r je definovaná ako $r \subseteq D_1 \times D_2 \times \dots \times D_n$. Jednotlivé prvky relácie r nazývame **tuples**, **záznamy** alebo **usporiadané *n*-tice**. Jednotlivým zložkám *n*-tíc hovoríme **atribúty** relácie. Množinám $D_1, D_2, \dots, D_n$ sa hovorí **domény** (alebo **typy**) atribútov relácie r .

Definícia 1.1.2. Jazyk logiky pozostáva z

1. predikátových symbolov
2. symbolov pre premenné
3. symbolov pre konštanty (reprezentujú prvky z domén)
4. logických symbolov ($\wedge, \neg$)
5. symbolov všeobecných kvantifikátorov ($\forall$)
6. symbolov pre definície ($\Rightarrow$)
7. čiarok (,)
8. zátvoriek ((,))

Definícia 1.1.3. Do **formúl** patria:

1. *atomické formuly* $p(t_1, t_2, \dots, t_n)$, kde p je *n*-árny predikát a $t_1, t_2, \dots, t_n$ sú konštanty alebo premenné
2. ak A a B sú formuly, tak potom aj $A \wedge B$ a $\neg A$ sú formuly
3. nič iné formula nie je.

Definícia 1.1.4. $A \Rightarrow B$ je **implikácia**, ak A je formula a B je atomická formula.

Definícia 1.1.5. $\forall X\iota$ sa nazýva **kvantifikovaná implikácia**, kde X je premenná a ι je implikácia. Ak je kvantifikovaná každá premenná v kvantifikovanej implikácii, nazývame ju **klauzula**.

Definícia 1.1.6. $K_1 \wedge K_2 \wedge \dots \wedge K_n$ nazývame **teóriou**, kde $K_1, K_2, \dots, K_n$ sú klauzuly.

Definícia 1.1.7. Ohodnotenie premenných je funkcia, ktorá priradí každej premennej konštantu. Ohodnotenie o , ktoré premennej X_i priradí konštantu k_i (pre $i \leq n$) označujeme $o(X_1|k_1, \dots, X_n|k_n)$. Keď vo formule A resp. implikácii ι resp. teórii $K_1 \wedge \dots \wedge K_n$ nahradíme všetky premenné podľa ohodnotenia o , výslednú formulu označujeme ako $A[o]$ resp. $\iota[o]$ resp. $K_1 \wedge \dots \wedge K_n[o]$.

Definícia 1.1.8. Interpretácia jazyka je funkcia, ktorá každej atomickej formule priradí jednu z hodnôt $\top, \perp, \Psi$.

Definícia 1.1.9. Nech $\mathcal{I}$ je interpretácia jazyka, φ je formula a o je ohodnotenie premenných. Potom $\mathcal{I} \models_{\top} \varphi[o]$ resp. $\mathcal{I} \models_{\perp} \varphi[o]$ resp. $\mathcal{I} \models_{\Psi} \varphi[o]$ označuje, že formula $\varphi[o]$ je pravdivá resp. nepravdivá resp. *unknown* vzhľadom na interpretáciu jazyka $\mathcal{I}$.

Pravdivosť formuly definujeme nasledovne:

1. $\mathcal{I} \models_{\top} \neg\varphi[o]$ ak $\mathcal{I} \models_{\perp} \varphi[o]$
 $\mathcal{I} \models_{\perp} \neg\varphi[o]$ ak $\mathcal{I} \models_{\top} \varphi[o]$
 $\mathcal{I} \models_{\Psi} \neg\varphi[o]$ ak $\mathcal{I} \models_{\Psi} \varphi[o]$
2. $\mathcal{I} \models_{\top} \varphi[o] \wedge \phi[o]$ ak $\mathcal{I} \models_{\top} \varphi[o]$ a zároveň $\mathcal{I} \models_{\top} \phi[o]$
 $\mathcal{I} \models_{\perp} \varphi[o] \wedge \phi[o]$ ak $\mathcal{I} \models_{\perp} \varphi[o]$ alebo $\mathcal{I} \models_{\perp} \phi[o]$
 $\mathcal{I} \models_{\Psi} \varphi[o] \wedge \phi[o]$ v ostatných prípadoch
3. $\mathcal{I} \models_{\top} (\varphi \Rightarrow \phi)[o]$ ak platí
 - (a) $\mathcal{I} \models_{\top} \varphi[o]$ a zároveň $\mathcal{I} \models_{\top} \phi[o]$
 - (b) $\mathcal{I} \models_{\Psi} \varphi[o]$ a zároveň $\mathcal{I} \models_{\top} \phi[o]$
 - (c) $\mathcal{I} \models_{\Psi} \varphi[o]$ a zároveň $\mathcal{I} \models_{\Psi} \phi[o]$
 - (d) $\mathcal{I} \models_{\perp} \varphi[o]$ $\mathcal{I} \models_{\perp} (\varphi \Rightarrow \phi)[o]$ ak $\mathcal{I} \models_{\top} \varphi[o]$ a zároveň $\mathcal{I} \models_{\perp} \phi[o]$
 $\mathcal{I} \models_{\Psi} (\varphi \Rightarrow \phi)[o]$ ak $\mathcal{I} \models_{\Psi} \varphi[o]$ a zároveň $\mathcal{I} \models_{\perp} \phi[o]$
4. Pre kvantifikovanú implikáciu $\forall X\iota$ platí:

$\mathcal{I} \models_{\top} \forall X\iota$ ak pre ľubovoľnú konštantu c platí $\mathcal{I} \models_{\top} \iota[X|c]$
 $\mathcal{I} \models_{\perp} \forall X\iota$ ak pre niektorú konštantu c platí $\mathcal{I} \models_{\perp} \iota[X|c]$
 inak $\mathcal{I} \models_{\Psi} \forall X\iota$

Definícia 1.1.10. Ak n -árny predikát p_r je prislúchajúci n -árnej relácii $r \subseteq D_1 \times D_2 \times \dots \times D_n$, potom v každej interpretácii $\mathcal{I}$ jazyka, ktorá obsahuje p_r , platí $\mathcal{I} \models_{\top} p_r(c_1, c_2, \dots, c_n)$ ak $(c_1, c_2, \dots, c_n) \in r$, inak $\mathcal{I} \models_{\perp} p_r(c_1, c_2, \dots, c_n)$.

Definícia 1.1.11. Interpretáciu $\mathcal{I}$ teórie $K_1 \wedge \dots \wedge K_n$ nazývame **model**, ak pre všetky $i \in \{1, \dots, n\}$ platí $\mathcal{I} \models_{\top} K_i$.

Definícia 1.1.12. Majme teóriu $K_1 \wedge \dots \wedge K_n$ a atomickú formulu Q . Potom teóriu $K_1 \wedge \dots \wedge K_n \wedge Q$ nazývame **teória s dotazom** Q alebo aj **program** resp. **datalogový program**.

Definícia 1.1.13. Majme teóriu s dotazom $K_1 \wedge \dots \wedge K_n \wedge Q$, kde $X_1, \dots, X_k$ sú premenné v dotaze Q . **Výsledkom dotazu** (vzhľadom na model $\mathcal{I}$ teórie $K_1 \wedge \dots \wedge K_n$) nazývame množinu usporiadaných k -tic

$$\{(c_1, c_2, \dots, c_k) \mid \mathcal{I} \models_{\top} Q[X_1|c_1, X_2|c_2, \dots, X_k|c_k]\}.$$

Ak dotaz Q neobsahuje žiadnu premennú a platí $\mathcal{I} \models_{\top} Q$, výsledkom dotazu je $\top$, inak je výsledkom $\perp$.

Poznámka 1.1.2. Môžeme si všimnúť, že dotazom sa vieme pýtať iba otázky typu *platí* $p(K_1, \dots, K_n)$?, kde $K_1, \dots, K_n$ sú konštanty. Nevieme sa pýtať na opak (otázky typu *neplatí*...?).

Týmto dokážeme odpovedať na niektoré dotazy pre teórie, ktoré nemajú dvojhodnotový model. Ako príklad môžeme zobrať teóriu

$$\mathcal{T} = (\neg p \Rightarrow q) \wedge (\neg q \Rightarrow p).$$

Jediná validná interpretácia $\mathcal{I}_V$ je $\mathcal{I}_V \models_{\top} p$ a $\mathcal{I}_V \models_{\top} q$ (ak by sme zobrali ľubovoľnú inú interpretáciu $\mathcal{I}$, došli by sme k sporu $\mathcal{I} \models_{\top} p \wedge \mathcal{I} \not\models_{\top} p$ resp. $\mathcal{I} \models_{\perp} p \wedge \mathcal{I} \not\models_{\perp} p$). Ak by sme sa vedeli pýtať na záporné dotazy, dostali by sme pre teórie s dotazom $\mathcal{T} \wedge p$ a $\mathcal{T} \wedge \neg p$ výsledok dotazu $\perp$. S našim prístupom dostaneme, že výsledkom dotazu „*platí* p ?“ je *false* ($\perp$).

Definícia 1.1.14. Klauzulu

$$\forall X_1 \forall X_2 \dots \forall X_n (A_1 \wedge A_2 \wedge \dots \wedge A_i \wedge \neg A_{i+1} \wedge \neg A_{i+2} \wedge \dots \wedge \neg A_m) \Rightarrow B$$

nazývame **bezpečnou**, ak sa každá z premenných $X_1, X_2, \dots, X_n$ vyskytuje v niektorej z pozitívnych atomických formúl $A_1, A_2, \dots, A_i$. Teóriu, pozostáva iba z bezpečných klauzúl (spojených konjunkciou), nazývame **bezpečnou teóriou**.

Poznámka 1.1.3. Vo zvyšku práce budeme pracovať iba s bezpečnými klauzulami a bezpečnými teóriami. Dôvodom je, že potom nemusíme vôbec uvažovať o doménach predikátov. Každá bezpečná formula je totiž doménovo nezávislá.

Zoberme si nebezpečnú (a doménovo závislú) teóriu s dotazom

$$\mathcal{T} = (\forall X \neg p(X) \Rightarrow q(X)) \wedge q(X)$$

s interpretáciou $\mathcal{I}$ takou, že $\forall i \in \mathbb{N} : \mathcal{I} \models_{\top} q(i)$ a pre všetky ostatné konštanty c platí $\mathcal{I} \models_{\perp} q(c)$. Čo je potom ale výsledok dotazu $\mathcal{T}$? Intuitívne by to malo byť všetko, čo nie je prirodzené číslo. Čo to ale je? Ak je doménou predikátu p napríklad množina celých čísel $\mathbb{Z}$, potom výsledkom dotazu bude množina $\mathbb{Z} - \mathbb{N}$ (t.j. množina všetkých záporných čísel). Ak by doménou bola množina racionálnych čísel $\mathbb{Q}$, výsledkom dotazu bude $\mathbb{Q} - \mathbb{N}$. Pri doménovo závislých formulách domény predikátov ovplyvňujú výsledky dotazov.

Príkladom bezpečnej teórie s dotazom je napríklad

$$\mathcal{T}' = (\forall X r(X) \wedge \neg p(X) \Rightarrow q(X)) \wedge p(X)$$

s interpretáciou $\mathcal{I}'$ takou, že $\forall i \in \mathbb{N} : \mathcal{I}' \models_{\top} p(i)$ a $\forall c \in \mathbb{R} : \mathcal{I}' \models_{\top} r(c)$. V tejto teórii sme obmedzili množinu konštánt, ktoré má zmysel dopĺňať do predikátu p . Výsledok tohoto dotazu je $\mathbb{R} - \mathbb{N}$, nezávisle od domén predikátov.

1.2 Rozšírenie datalogu o funkčné symboly

Doteraz sme dosádzali do predikátov iba konštanty a premenné. Rozšírime jazyk logiky o tzv. funkčné symboly. Funkčný symbol f (s aritou n) reprezentuje ľubovoľnú funkciu $f : D_1 \times D_2 \times \dots \times D_n \rightarrow D$.

Funkčné symboly chceme zaviesť preto, že samotné konštanty a premenné nestačia na reprezentáciu zložitejších štruktúr. Príkladom zložitejšej štruktúry je zoznam. Zoznam vieme reprezentovať pomocou funkčných symbolov `nil` a `list`: `nil`, ktorý reprezentuje prázdny zoznam, a `list`, ktorého prvým argumentom je prvý prvok zoznamu, a druhým argumentom je zvyšok zoznamu. Zoznam obsahujúci prvky a, b a c (v tomto poradí) potom bude reprezentovať termom `list(a, list(b, list(c, nil)))`.

Definícia 1.2.1. Za **term** považujeme:

- ľubovoľný 0-árny funkčný symbol (ktorý považujeme za *konštantu*, môžeme zapisovať aj bez zátvoriek),
- ľubovoľnú premennú (budeme značiť veľkými písmenami latinskej abecedy),
- ak f je n -árny funkčný symbol a $t_1, t_2, \dots, t_n$ sú termy, potom $f(t_1, t_2, \dots, t_n)$ je term.

Dôležité je, že term pozostáva z konečného množstva symbolov.

Rozšírime definíciu jazyka logiky (1.1.2) o symboly označujúce funkčné symboly. Do predikátov (1.1.3 1.) potom nedosadzujeme premenné a konštanty, ale termy.

Definícia 1.2.2. **Ground term** je ľubovoľný term, v ktorom sa nevyskytujú premenné.

V definícii pre ohodnotenie premenných (1.1.7) sa daná funkcia predefinuje na funkcii, ktorá priradí každej premennej nie konštantu, ale *ground term*. Taktiež v definícii interpretácie jazyka (1.1.9) prepíšeme 4. časť na:

Pre kvantifikovanú implikáciu $\forall X\iota$ platí:

$\mathcal{I} \models_{\top} \forall X\iota$ ak pre *ľubovoľný ground term* c platí $\mathcal{I} \models_{\top} \iota[X|c]$,

$\mathcal{I} \models_{\perp} \forall X\iota$ ak pre *niektorý ground term* c platí $\mathcal{I} \models_{\perp} \iota[X|c]$,

inak $\mathcal{I} \models_{\bot} \forall X\iota$.

Ostatné definície môžu ostať tak, ako boli popísané v predošlej kapitole.

1.3 Vstavané predikáty

Klasický datalog povoľuje prácu iba s predikátmi definovanými v programe, či už pomocou faktov alebo pravidiel. V prípade, že chceme pracovať s veľkou databázou s niekoľkými tabuľkami, museli by sme pre každú tabuľku vytvoriť predikát a naplniť ho faktami v každom programe, v ktorom chceme s danou tabuľkou pracovať. To by bolo veľmi neefektívne a nepraktické. Preto zavedieme tzv. **vstavané predikáty**. Tie budú práve reprezentovať veľké tabuľky alebo predikáty, ktoré by bolo nepraktické definovať a implementovať v datalogu. Z teoretického hľadiska môžu byť ľubovoľné, kľudne aj nekonečné alebo nevypočítateľné. V praxi však budú vstavané predikáty implementované v samotnom databázovom systéme, ktorý je v našom prípade naprogramovaný v jazyku Java. Keďže budú implementované vo vyššom programovacom jazyku ako datalog, môžu pracovať aj so zložitejšími vstupmi, ako s inými predikátmi, termami a zoznamami. Vieme ich použiť v pravidlách rovnako ako bežné predikáty s jedným obmedzením. Nemôžeme ich predefinovať alebo dedefinovať, čiže sa nemôžu vyskytovať na pravej strane implikácie. Vstavané predikáty budeme označovať pomocou \$ pred menom predikátu.

Definícia 1.3.1. Pod pojmom **parameter** rozumieme:

1. ľubovoľný term
2. ľubovoľný predikát alebo vstavaný predikát
3. ľubovoľný zoznam parametrov, ktorý zapisujeme ako $[P_1, P_2, \dots, P_n]$,
kde $P_1, P_2, \dots, P_n$ sú parametre

Definícia 1.3.2. Atomické formuly sú

1. $p(t_1, t_2, \dots, t_n)$, kde p je n -árny predikát a $t_1, t_2, \dots, t_n$ sú termy
2. $\$p(P_1, P_2, \dots, P_n)$, kde $\$p$ je n -árny vstavaný predikát a $P_1, P_2, \dots, P_n$ sú parametre

1.4 Syntax rozšíreného datalogu

Zoberme si formálny zápis relatívne jednoduchej teórie s dotazom:

$$\begin{aligned} & [\forall X, Y (\text{parent}(X, Y) \Rightarrow \text{ancestor}(X, Y))] \wedge \\ & [\forall X, Y, Z ((\text{parent}(X, Z) \wedge \text{ancestor}(Z, Y)) \Rightarrow \text{ancestor}(X, Y))] \wedge \\ & \text{ancestor}(X, Y) \end{aligned}$$

Príklad 1.1

Tento zápis sa rýchlo stáva neprehľadným a dlhým. Preto Datalog používa skrátený zápis.

Klauzula

$$\forall X_1, \dots, X_k ((P_1 \wedge \dots \wedge P_i \wedge \neg P_{i+1} \wedge \dots \wedge \neg P_n) \Rightarrow Q),$$

kde $X_1, \dots, X_k$ sú všetky premenné vyskytujúce sa v termoch použitých v predikátoch $P_1, \dots, P_n$ a Q , sa v skrátrenom zápise zapisuje ako

$$Q \leftarrow P_1, \dots, P_i, \neg P_{i+1}, \dots, \neg P_n.$$

Teórii s dotazom $K_1 \wedge K_2 \wedge \dots \wedge K_n \wedge Q$ zodpovedá $D_1 D_2 \dots D_n ?Q$, kde D_i je skrátený zápis klauzuly K_i .

Skôr spomenutá teória je reprezentovaná skráteným zápisom:

$$\begin{aligned} & \text{ancestor}(X, Y) \leftarrow \text{parent}(X, Y). \\ & \text{ancestor}(X, Y) \leftarrow \text{parent}(X, Z), \text{ancestor}(Z, Y). \\ & ? \text{ancestor}(X, Y) \end{aligned}$$

Príklad 1.2

Poznámka 1.4.1. So znakom **t**, resp. **f** budeme označovať 0-árny predikát, pre ktorý pre každú interpretáciu $\mathcal{I}$ platí $\mathcal{I} \models_{\top} \mathbf{t}$, resp. $\mathcal{I} \models_{\perp} \mathbf{f}$.

Definícia 1.4.1. Klausulu typu $p(t_1, \dots, t_n) \leftarrow \mathbf{t}$, kde p je n -árny predikát a $t_1, \dots, t_n$ sú termy, nazývame **fakt** alebo **definícia**. Klasicky ich skrátene zapisujeme ako $p(t_1, \dots, t_n)$. (bodka na konci je súčasťou skráteného zápisu). Keďže ale často potrebujeme vyjadriť pre jeden predikát viacero faktov, zvolíme nasledujúci zápis. Fakty

$$p(t_{1,1}, \dots, t_{1,n}).$$

$$\vdots$$

$$p(t_{k,1}, \dots, t_{k,n}).$$

zapišeme ako

$$p \leftarrow \{(t_{1,1}, \dots, t_{1,n}), \dots, (t_{k,1}, \dots, t_{k,n})\}.$$

(zdôrazňujem, že bodka na konci je súčasťou zápisu).

V ASCII zápise datalogových programov a dotazov sa používajú alternatívne znaky, konkrétne:

- $\leftarrow \rightsquigarrow :-$
- $\neg \rightsquigarrow \setminus +$
- Na označenie dotazov sa namiesto $?$ používa $?-$.

Doteraz spomínaná teória (rozšírená o fakty) by potom bola zapísaná ako:

```
ancestor :- {(jozo, fero), (fero, sano)}.
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
?- ancestor(X, Y).
```

Príklad 1.3

1.5 Sémantika datalogu

Zoberme si veľmi jednoduchú teóriu:

$$p = \{(1)\}.$$

Pre túto teóriu existuje niekoľko modelov (dokonca ich je nekonečne veľa). Najjednoduchší z nich je $\mathcal{I} \models_{\top} p(1)$ a pre všetky ostatné formuly je interpretácia nepravdivá. Ďalšími modelmi môžu byť napríklad:

- $\mathcal{I}' \models_{\top} p(i), \forall i \in \mathbb{N}$ a pre všetky ostatné formuly je interpretácia nepravdivá,
- $\mathcal{I}'' \models_{\top} p(i), \forall i \in \mathbb{R}$ a pre všetky ostatné formuly je interpretácia nepravdivá,
- $\mathcal{I}''' \models_{\top} p(1), \mathcal{I}''' \models_{\top} p(\text{Pes}), \mathcal{I}''' \models_{\top} p(47), \mathcal{I}''' \models_{\top} p(\text{Štvorec})$ a pre všetky ostatné formuly je interpretácia nepravdivá.

Ako sémantiku našich datalogových programov budeme uvažovať *well-founded model*. Well-founded model je minimálny stabilný trojhodnotový model.

Trojhodnotový model znamená, že interpretácia priradzuje atomickým formulám hodnoty *true* $\top$, *false* $\perp$ a *unknown* $\sqcup$.

Stabilný model znamená, že zostane rovnaký po tzv. *Gelfond-Lifschitz Transformation*. Aplikácia GLT funguje nasledovne: zoberieme aktuálnu interpretáciu a pomocou nej z programu odstránime tie pravidlá, ktorých negované podmienky sú podľa danej interpretácie pravdivé. Z ostatných pravidiel potom odstránime negácie. Tým dostaneme program bez negácie, pre ktorý vieme zostrojiť minimálny model. Ak sa tento minimálny model zhoduje s interpretáciou, z ktorej sme vychádzali, hovoríme, že ide o stabilný model.

Zoberme si program

- a.
- b :- \+ c.
- c :- \+ b.
- d :- a, \+ c.

a kandidátnu interpretáciu $\mathcal{I}$, pre ktorú platí

$$\mathcal{I} \models_{\top} a, \mathcal{I} \models_{\top} b, \mathcal{I} \models_{\perp} c, \mathcal{I} \models_{\top} d.$$

Pri aplikácii GLT najprv odstránime tie pravidlá, ktorých negované podmienky sú podľa $\mathcal{I}$ pravdivé. Pravidlo c :- \+ b. sa preto odstráni, keďže b je v interpretácii $\mathcal{I}$ pravdivé. V ostatných pravidlách potom odstránime negácie, a dostaneme program

- a.
- b.
- d :- a.

Minimálny model takto upraveného programu obsahuje práve atómy a, b a d. Keďže sa zhoduje s interpretáciou, z ktorej sme vychádzali, daná interpretácia je stabilný model pôvodného programu.

Minimálny model znamená, že spomedzi všetkých modelov vyberáme ten, ktorý obsahuje najmenej pravdivých atómov. Intuitívne teda nechceme odvodiť viac faktov, než je nutné. Ak existuje viac modelov, uprednostníme ten najmenší.

Príklad minimálneho modelu pre teóriu

$$p = \{(1)\}.$$

je modelom napríklad interpretácia, v ktorej platí iba $p(1)$. Modelom je aj interpretácia, v ktorej platí $p(1)$, $p(2)$ a $p(3)$. Minimálny model je však ten prvý, pretože obsahuje najmenej pravdivých atómov.

Well-founded model môžeme chápať ako niečo, čo je spoločné pre všetky stabilné modely. Atómy, ktoré sú pravdivé vo všetkých stabilných modeloch, označíme ako $\top$, atómy, ktoré sú nepravdivé vo všetkých stabilných modeloch, označíme ako $\perp$, a atómy, pri ktorých sa stabilné modely rozchádzajú, ponecháme ako Ψ .

V nasledujúcom príklade ukážeme, ako sa dá nájsť well-founded model teórie. Nejde o formálny dôkaz. Pre viac o well-founded model odporúčam prečítať [9] a [4].

Budeme postupovať iteratívne. Začneme interpretáciou, v ktorej sú všetky atómy nepravdivé, a budeme opakovane aplikovať GLT. Po každom kroku vezmeme minimálny model získaného programu a ten použijeme ako novú interpretáciu. Keď sa interpretácia prestane meniť, dostaneme výsledný model.

Príklad nájdenia well-founded modelu:

```
a.
b :- a.
p :- \+ q.
q :- \+ p.
u :- v.
```

Budeme uvažovať atómy a , b , p , q , u a v . Na začiatku zoberieme interpretáciu $\mathcal{I}_0$, v ktorej sú všetky tieto atómy nepravdivé.

Pri aplikácii GLT na $\mathcal{I}_0$ sa neodstráni žiadne pravidlo, pretože v $\mathcal{I}_0$ nie je pravdivý ani p , ani q . Po odstránení negácií dostaneme program

```
a.
b :- a.
p.
q.
u :- v.
```

Jeho minimálny model je interpretácia $\mathcal{I}_1$, v ktorej platí

$$\mathcal{I}_1 \models_{\top} a, \mathcal{I}_1 \models_{\top} b, \mathcal{I}_1 \models_{\top} p, \mathcal{I}_1 \models_{\top} q, \mathcal{I}_1 \models_{\perp} u, \mathcal{I}_1 \models_{\perp} v.$$

Teraz aplikujeme GLT na interpretáciu $\mathcal{I}_1$. Keďže v nej sú p aj q pravdivé, pravidlá $p :- \neg q$. aj $q :- \neg p$. sa odstránia. Po odstránení negácií dostaneme program

a.
b :- a.
u :- v.

Jeho minimálny model je interpretácia $\mathcal{I}_2$, v ktorej platí

$$\mathcal{I}_2 \models_{\top} a, \mathcal{I}_2 \models_{\top} b, \mathcal{I}_2 \models_{\perp} p, \mathcal{I}_2 \models_{\perp} q, \mathcal{I}_2 \models_{\perp} u, \mathcal{I}_2 \models_{\perp} v.$$

Ak teraz ešte raz aplikujeme GLT na interpretáciu $\mathcal{I}_2$, obe pravidlá s negáciou opäť zostanú, a preto znovu dostaneme program

a.
b :- a.
p.
q.
u :- v.

Jeho minimálny model je opäť $\mathcal{I}_1$. Dostali sme teda cyklus

$$\mathcal{I}_0 \rightarrow \mathcal{I}_1 \rightarrow \mathcal{I}_2 \rightarrow \mathcal{I}_3 \equiv \mathcal{I}_1 \rightarrow \mathcal{I}_2 \rightarrow \dots$$

Vidíme, že atómy a a b sú pravdivé v každom kroku od $\mathcal{I}_1$ ďalej, a preto im vo well-founded modeli priradíme hodnotu $\top$. Atómy u a v nie sú pravdivé v žiadnom kroku, a preto im priradíme hodnotu $\perp$. Naopak, pri atómoch p a q sa interpretácie striedajú: raz sú pravdivé a raz nepravdivé. Tým pádom ich nevieme jednoznačne určiť, a vo well-founded modeli dostanú hodnotu $\perp$.

Výsledný well-founded model tejto teórie je teda daný hodnotami

$$a = \top, \quad b = \top, \quad u = \perp, \quad v = \perp, \quad p = \perp, \quad q = \perp.$$

Prehľadne to môžeme zapísať aj do tabuľky:

Atóm	$\mathcal{I}_0$	$\mathcal{I}_1$	$\mathcal{I}_2$	$\mathcal{I}_3$	$\mathcal{I}$
a	$\perp$	$\top$	$\top$	$\top$	$\top$
b	$\perp$	$\top$	$\top$	$\top$	$\top$
u	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
v	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
p	$\perp$	$\top$	$\perp$	$\top$	$\perp$
q	$\perp$	$\top$	$\perp$	$\top$	$\perp$

Kapitola 2

Relačná algebra

2.1 Definície

V tejto časti sa inšpirujeme relačnou algebrou z [2] a klasickou *učebnicovou* relačnou algebrou a definujeme našu vlastnú.

2.1.1 Učebnicová relačná algebra

Klasická relačná algebra pracuje s reláciami, ktoré majú väčšinou jednotlivé atribúty pomenované. Potom ich vieme jednoducho reprezentovať pomocou tabuliek.

$R \in \mathbf{Meno} \times \mathbf{Výška}$ $R = \{(\text{Adam}, 175), (\text{Boris}, 183), (\text{Cecil}, 192)\}$	R	
	Meno	Výška
	Adam	175
	Boris	183
	Cecil	192

Príklad 2.1

Na manipuláciu s týmito reláciami existuje niekoľko operátorov:

Množinové operátory

zjednotenie množín $\cup$, rozdiel množín $-$, (prienik množín $\cap$, ale ten sa dá vyjadriť aj za pomoci rodielu $A \cap B = A - (A - B)$)

Pre relácie, na ktoré aplikujeme tieto operátory musí platiť, že majú identické domény atribútov. Pred tým, ako aplikujeme operátor na dve relácie, musíme usporiadať ich atribúty v jednotlivých záznamoch do rovnakého poradia.

Karteziánsky súčin $\times$

$R \times S$, kde R a S sú relácie. Výsledná relácia sa dá matematicky zapísať nasledovne:

$$R \times S = \{(a_1, \dots, a_n, b_1, \dots, b_m) \mid (a_1, \dots, a_n) \in R, (b_1, \dots, b_m) \in S\}$$

		$R \times S$			
		X	R.Y	S.Y	Z
R					
X	Y				
Y	Z				
A	a	1	10	A	a
B	b	2	20	B	b
C	c	3	30	C	c
		A	a	2	20
		B	b	2	20
		C	c	2	20
		A	a	3	30
		B	b	3	30
		C	c	3	30

Príklad 2.2

Projekcia Π

$\Pi_{a_1, \dots, a_n}(R)$, kde $a_1, \dots, a_n$ sú názvy atribútov relácie R , vráti novú reláciu, ktorá obsahuje „iba stĺpce“ $a_1, \dots, a_n$. Výsledok je stále relácia (čiže aj množina), takže neobsahuje duplicitné záznamy.

R			$\Pi_{\text{Meno}, \text{Zviera}}(R)$		$\Pi_{\text{Mesto}}(R)$
Meno	Mesto	Zviera	Meno	Zviera	Mesto
Adam	Bratislava	Pes	Adam	Pes	Bratislava Košice Lučenec Poprad
Boris	Bratislava	Mačka	Boris	Mačka	
Cecil	Košice	Pes	Cecil	Pes	
Dano	Lučenec	Zajac	Dano	Zajac	
Erik	Poprad	Kôň	Erik	Kôň	
Fero	Košice	Mačka	Fero	Mačka	

Príklad 2.3

Selekcia σ

$\sigma_C(R)$, kde C je podmienka, ktorá obsahuje atribúty relácie R . Výsledná relácia je podmnožina relácie R obsahujúca iba tie záznamy, ktoré spĺňajú podmienku C .

R			$\sigma_{\text{Mesto}=\text{Bratislava} \vee \text{Zviera}=\text{Mačka}}(R)$		
Meno	Mesto	Zviera	Meno	Mesto	Zviera
Adam	Bratislava	Pes	Adam	Bratislava	Pes
Boris	Bratislava	Mačka	Boris	Bratislava	Mačka
Cecil	Košice	Pes	Fero	Košice	Mačka
Dano	Lučenec	Zajac			
Erik	Poprad	Kôň			
Fero	Košice	Mačka			

Príklad 2.4

Prirodzený (natural) join $\bowtie$

$R \bowtie S$, kde R a S sú relácie. Podobne ako v karteziánskom súčine párujeme záznamy z dvoch relácií, ale iba tie záznamy z R a S , ktoré sa zhodujú na atribútoch spoločných pre R a S . V množinovom zápise:

$$\begin{aligned}
 R &= A_1 \times \cdots \times A_n \times C_1 \times \cdots \times C_k \\
 S &= B_1 \times \cdots \times B_n \times C_1 \times \cdots \times C_k \\
 R \times S &= \{(a_1, \dots, a_n, c_1, \dots, c_k, b_1, \dots, b_m) \mid \\
 &\quad (a_1, \dots, a_n, c_1, \dots, c_k) \in R, \\
 &\quad (b_1, \dots, b_m, c'_1, \dots, c'_k) \in S, \\
 &\quad c_1 = c'_1, \dots, c_k = c'_k\}
 \end{aligned}$$

R		S		$R \bowtie S$		
Meno	Mesto	Mesto	Rozloha	Meno	Mesto	Rozloha
Adam	Bratislava	Bratislava	368	Adam	Bratislava	368
Boris	Bratislava	Košice	242	Boris	Bratislava	368
Cecil	Košice	Lučenec	47	Cecil	Košice	242
Dano	Lučenec	Poprad	63	Dano	Lučenec	47
Erik	Poprad			Erik	Poprad	63
Fero	Košice			Fero	Košice	242

Príklad 2.5

Antijoin $\triangleright$

$R \triangleright S$, kde R a S sú relácie. Vo výslednej relácii sú tie záznamy z R , pre ktoré neexistuje záznam v S , s ktorým by sa zhodovali na spoločných atribútoch. V množinovom zápise:

$$R = A_1 \times \cdots \times A_n \times C_1 \times \cdots \times C_k$$

$$S = B_1 \times \cdots \times B_n \times C_1 \times \cdots \times C_k$$

$$R \times S = \{(a_1, \dots, a_n, c_1, \dots, c_k) \in R \mid \forall (b_1, \dots, b_n, c'_1, \dots, c'_k) \in S : c_1 \neq c'_1, \dots, c_k \neq c'_k\}$$

Z			
Meno	Mesto		
Adam	Bratislava	V	$Z \triangleright V$
Boris	Bratislava		
Cecil	Košice	Mesto	Meno
Dano	Lučenec	Bratislava	Cecil
Erik	Poprad	Lučenec	Erik
Fero	Košice		Fero

Príklad 2.6

Theta-join $\bowtie_C$

$R \bowtie_C S$, kde R a S sú relácie a C je podmienka, v ktorej sa vyskytujú atribúty z R a S . Natural join nám povolil spojiť dva záznamy iba ak ich spoločné atribúty boli rovnaké. Theta-join popáruje každú dvojicu záznamov z R a S , ktoré spĺňajú podmienku C .

M		R			$M \bowtie_{R.Od < M.Roz \wedge M.Roz < R.Od} R$				
Mesto	Roz	Od	Do	Veľ	Mesto	Roz	Od	Do	Veľ
Bratislava	368	0	100	Malé	Bratislava	368	301	1000	Veľké
Košice	242	101	300	Stredné	Košice	242	101	300	Stredné
Lučenec	47	301	1000	Veľké	Lučenec	47	0	100	Malé
Poprad	63				Poprad	63	0	100	Malé

Príklad 2.7

Premenovanie ρ

$\rho_{A/B}(R)$, kde R je relácia, A je názov atribútu z relácie R a B je nový názov atribútu. Výsledkom bude relácia s rovnakými záznamami, iba atribút A sa premenuje na B .

R			$\rho_{\text{Mesto/Obec}}(R)$		
Meno	Mesto	Zviera	Meno	Obec	Zviera
Adam	Bratislava	Pes	Adam	Bratislava	Pes
Boris	Bratislava	Mačka	Boris	Bratislava	Mačka
Cecil	Košice	Pes	Cecil	Košice	Pes
Dano	Lučenec	Zajac	Dano	Lučenec	Zajac
Erik	Poprad	Kôň	Erik	Poprad	Kôň
Fero	Košice	Mačka	Fero	Košice	Mačka

Príklad 2.8

Iné join-y

V niektorých knihách sa spomínajú aj iné variácie join-ov ako *left outer join*, *right outer join* alebo *full outer join*. My sa s nimi nebudeme zaoberať, keďže sa dajú vyskladať z predchádzajúcich operátorov. Pre ich správne fungovanie treba ale zaviesť špeciálnu konštantu, ktorá sa v literatúre označuje ako ω , ε , `null` alebo `none`.

2.1.2 Naša relačná algebra

Pre jednoduchší preklad datalogu do relačnej algebry sme si zvolili nasledujúcu algebru so štyrmi operátormi (inšpirovaná algebrou z [2]). Tiež pracuje s reláciami, ale nepotrebuje vopred pomenované atribúty. Vždy, keď potrebuje pracovať s „názvami stĺpcov“, sú explicitne vymenované.

UNION ($[R_1, \dots, R_n]$), kde $R_1, \dots, R_n$ sú relácie s rovnakým počtom atribútov. Výsledná relácia bude obsahovať všetky záznamy z každej relácie R_1 až R_n (až na duplikáty). Formálne $\text{UNION}([R_1, \dots, R_n]) = \bigcup_{i=1}^n R_i$.

			UNION([A, B, C])	
			11	a
			12	b
			13	c
A	B	C	21	a
11	a	21	a	a
12	b	22	b	b
13	c	23	c	c
			31	a
			32	b
			33	c

Príklad 2.9

JOIN([$R_1, \dots, R_n$], [[$T_{1,1}, \dots, T_{1,a_1}$], $\dots$, [$T_{n,1}, T_{n,a_n}$]], [$T_1, \dots, T_k$]), kde R_i je relácia s aritou a_i , $T_{i,j}$ sú vstupné termy a T_l sú výstupné termy.

Výsledná relácia bude nasledujúca. Nech $P_1, \dots, P_p$ sú všetky premenné vyskytujúce sa v termoch $T_{1,1}, \dots, T_{1,a_1}, T_{2,1}, \dots, T_{n,a_n}$. Nech množina M obsahuje všetky $(c_1, \dots, c_p)$ také, pre ktoré platí

$$(T_{i,1}[P_1|c_1, \dots, P_p|c_p], \dots, T_{i,a_i}[P_1|c_1, \dots, P_p|c_p]) \in R_i, \forall i \in \{1, \dots, n\}.$$

Potom vo výslednej relácii R budú záznamy

$$R = \{(T_1[P_1|k_1, \dots, P_p|k_p], \dots, T_k[P_1|k_1, \dots, P_p|k_p]) \mid (k_1, \dots, k_p) \in M\}.$$

G		JOIN([G, G], [[X, Z], [Z, Y]], [X, Y])	
1	2	1	3
2	3	2	4
3	4	2	5
3	5		

Príklad 2.10

ANTIJOIN([$R_1, \dots, R_n$], [[$T_{1,1}, \dots, T_{1,a_1}$], $\dots$, [$T_{n,1}, T_{n,a_n}$]], [$T_1, \dots, T_k$]), kde R_i je relácia s aritou a_i , $T_{i,j}$ sú vstupné termy a T_l sú výstupné termy.

Výsledná relácia bude nasledujúca. Nech $P_1, \dots, P_p$ sú všetky premenné vyskytujúce sa v termoch $T_{1,1}, \dots, T_{1,a_1}, T_{2,1}, \dots, T_{n,a_n}$. Nech množina M obsahuje všetky $(c_1, \dots, c_p)$ také, pre ktoré platí

$$(T_{1,1}[P_1|c_1, \dots, P_p|c_p], \dots, T_{1,a_1}[P_1|c_1, \dots, P_p|c_p]) \in R_1$$

a zároveň pre žiadne $i \in \{2, \dots, n\}$ neplatí

$$(T_{i,1}[P_1|c_1, \dots, P_p|c_p], \dots, T_{i,a_i}[P_1|c_1, \dots, P_p|c_p]) \in R_i.$$

Potom vo výslednej relácii R budú záznamy

$$R = \{(T_1[P_1|k_1, \dots, P_p|k_p], \dots, T_k[P_1|k_1, \dots, P_p|k_p]) \mid (k_1, \dots, k_p) \in M\}.$$

Nech f je funkčný symbol, ktorý pre ľubovoľný reťazec vráti jeho prvý znak.

U			
Adam	Bratislava		ANTIJOIN ($[U, P], [[M, B], [f(B)]], [M]$)
Boris	Bratislava	P	Adam
Cecil	Košice	B	Dano
Dano	Lučenec	K	Erik
Erik	Poprad		Fero
Fero	Košice		

Príklad 2.11

REC(N, R), kde N je názov relácie a R je relácia. Tento operátor definuje reláciu s názvom N a dovoľuje použiť v relácii R odkaz na ňu.

$$\mathbf{REC}(t, \mathbf{UNION}([\$s, \mathbf{JOIN}([t, t], [[X, Z], [Z, Y]], [X, Y]])))$$

Relácia t je tranzitívny uzáver vstavanej relácie s .

Príklad 2.12

Vstavane operátory

Ako sme si ukázali skorej (definícia 1.1.10), na predikáty a relácie sa vieme pozeráť ako na rovnaké objekty. Taktiež na operátory **JOIN**, **ANTIJOIN**, **UNION** a **REC** sa vieme pozeráť ako na relácie, keďže ich výstupom je vždy relácia. Preto ku každému

vstavanému predikátu vieme definovať vstavaný operátor, ktorý bude fungovať rovnako a bude mať rovnaké vstupy. Jediný rozdiel bude formálny, keďže vstavané operátory budú mať výstupom reláciu, zatiaľ čo vstavané predikáty majú výstupom pravdivostnú hodnotu. Takýto prístup si ale vyžadoval zmenu parsera relačnej algebry [2].

Definícia 2.1.1. Parameter (*v kontexte relačnej algebry*) je:

1. ľubovoľný term
2. ľubovoľnú *reláciu* alebo vstavaný operátor (ktorého výstupom je relácia)
3. ľubovoľný zoznam parametrov, ktorý zapisujeme ako $[P_1, P_2, \dots, P_n]$,
kde $P_1, P_2, \dots, P_n$ sú parametre

Definícia 2.1.2. Operátor je $\$O(P_1, P_2, \dots, P_n)$, kde $\$O$ je názov operátora a $P_1, P_2, \dots, P_n$ sú parametre operátora. Výstupom operátora je relácia.

Starý prístup spracovával **JOIN**, **ANTIJOIN**, **UNION** a **REC** osobitne. Nový prístup ich spracováva rovnako ako ostatné operátory. Viac o ňom sa viete dočítať v [3]. Formálne by sme mali naše 4 operátory zapisovať s prefixom \$, ale keďže zvyšok implementácie vyžaduje, aby boli implementované v databázovom systéme tak, ako sme ich definovali, tak ich budeme zapisovať bez \$.

2.2 Inklúzia medzi učebnicovou a našou relačnou algebrou

V nasledujúcej kapitole sa pokúsime ukázať, že naša relačná algebra má aspoň takú výpočtovú silu ako učebnicová relačná algebra. To znamená, že každý výraz v učebnicovej relačnej algebre môžeme vyjadriť pomocou výrazov v našej relačnej algebre. Dôkazy nebudú úplne formálne správne, ale mali by byť dostatočne jasné a zrozumiteľné, aby bolo zrejmé, ako by sa mal ľubovoľný program v učebnicovej relačnej algebre prepísať do našej algebry. Pre viac informácií o našej relačnej algebre odporúčame pozrieť [2] a [3].

Množinové operátory

Zjednotenie množín (relácií) $R \cup S$ vieme prepísať ako **UNION** ($[R, S]$). Rozdiel množín $R - S$ prepíšeme ako

$$\mathbf{ANTIJOIN}([R, S], [[A_1, A_2, \dots, A_n], [A_1, A_2, \dots, A_n]], [A_1, A_2, \dots, A_n]).$$

Keďže prienik vieme vyjadriť pomocou rozdielu, budeme ho vedieť zapísať pomocou **ANTIJOIN**.

Karteziánsky súčin $\times$

Karteziánsky súčin $R \times S$ vieme prepísať pomocou **JOIN**. Jednoducho vymenujeme všetky atribúty z R a S ako vstupné termy a výstupné termy.

$$R \times S \approx \mathbf{JOIN}([R, S], \\ [[A_1, A_2, \dots, A_n], [B_1, B_2, \dots, B_m]], \\ [A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m])$$

Projekcia Π

Projekcia je už zakomponovaná v **JOIN** a **ANTIJOIN**. Ak ale chceme vyjadriť iba projekciu, môžeme použiť **JOIN** s pomenovanými vstupnými termami a vymenovanými výstupnými termami, ktoré sú podmnožinou vstupných termov. Napríklad $\Pi_{A_1, A_3}(R)$ prepíšeme ako

$$\mathbf{JOIN}([R], [[A_1, A_2, A_3, \dots, A_n]], [A_1, A_3]).$$

Treba si ale dávať pozor, že v našej algebre záleží na poradí atribútov a v učebnicovej algebre záleží iba na ich názvoch. Preto je rozdiel medzi

$$\mathbf{JOIN}([R], [[A_1, A_2, A_3]], [A_1, A_3]),$$

$$\mathbf{JOIN}([R], [[A_1, A_3, A_2]], [A_1, A_3]) \text{ a}$$

$$\mathbf{JOIN}([R], [[A_1, A_2, A_3]], [A_3, A_1]).$$

Selekcia σ

Pre selekciu využijeme veľkú výpočtovú silu funkčných symbolov. Zoberme si selekciu $\sigma_C(R)$. Skonstruujeme funkčný symbol f_C tak, že $f_C(p_1, p_2, \dots, p_n) = \mathbf{t}$ práve vtedy, keď podmienka C platí pre záznam $(p_1, p_2, \dots, p_n)$. Pre všetky ostatné vstupy bude f_C vracieť $\mathbf{f}$. Potom môžeme selekciu $\sigma_C(R)$ prepísať ako

$$\mathbf{JOIN}([R, \$true], [[p_1, p_2, \dots, p_n], [f_C(p_1, p_2, \dots, p_n)]], [p_1, p_2, \dots, p_n, \mathbf{t}]).$$

Výsledná relácia má ale o jeden atribút viac, preto ju treba ešte *obaliť* projekciou (joinom), ktorá nám tento atribút odstráni.

Prirodzený (natural) join $\bowtie$

Opäť si musíme dávať pozor na poradie atribútov. Preklad $R \bowtie S$ bude ale potom priamočiary. Ukážeme na príklade, keď relácie R a S majú spoločné atribúty $C_1, C_2, \dots, C_k$

na konci.

$$R \bowtie S \approx \mathbf{JOIN}([R, S], \\ [[A_1, \dots, A_n, C_1, \dots, C_k], [B_1, \dots, B_m, C_1, \dots, C_k]], \\ [A_1, \dots, A_n, C_1, \dots, C_k, B_1, \dots, B_m])$$

Antijoin $\triangleright$

Antijoin $R \triangleright S$ prepíšeme v podstate rovnako ako natural join. Znovu predpokladáme, že spoločné atribúty $C_1, C_2, \dots, C_k$ sú *na konci*.

$$R \triangleright S \approx \mathbf{ANTIJOIN}([R, S], \\ [[A_1, \dots, A_n, C_1, \dots, C_k], [B_1, \dots, B_m, C_1, \dots, C_k]], \\ [A_1, \dots, A_n, C_1, \dots, C_k])$$

Theta-join $\bowtie_C$

Ako pri selekcii, aj tu využijeme funkčné symboly. Zoberme si theta-join $R \bowtie_C S$. Skonstruujeme funkčný symbol f_C tak, že $f_C(p_1, p_2, \dots, p_n, q_1, q_2, \dots, q_m) = \mathbf{t}$ práve vtedy, keď podmienka C platí pre záznam $(p_1, p_2, \dots, p_n)$ z relácie R a záznam $(q_1, q_2, \dots, q_m)$ z relácie S . Pre všetky ostatné vstupy bude f_C vracaať $\mathbf{f}$. Potom dokážeme theta-join $R \bowtie_C S$ prepísať ako

$$\mathbf{JOIN}([R, S, \$true], \\ [[A_1, \dots, A_n], [B_1, \dots, B_m], [f_C(A_1, \dots, A_n, B_1, \dots, B_m)]], \\ [A_1, \dots, A_n, B_1, \dots, B_m, \mathbf{t}])$$

Výsledná relácia má ale znovu o jeden atribút viac, preto ju znovu treba *obaliť* projekciou (joinom), pomocou ktorej tento atribút odstránime.

Premenovanie ρ

Keďže v **JOIN** a **ANTIJOIN** zakaždým explicitne vymenujeme všetky vstupné a výstupné termy, premenovanie ρ nepredstavuje žiaden problém.

Kapitola 3

Kompilátor

3.1 ANTLR

Náš kompilátor používa generátor *parserov* **AN**other **T**ool for **L**anguage **R**ecognition. V tejto časti si priblížime, ako funguje *ANTLR* a ako funguje samotný preklad do relačnej algebry.

3.1.1 Rýchlokurz formálnych jazykov

Definícia 3.1.1. Pod **abecedou** rozumieme ľubovoľnú *neprázdnu konečnú* množinu symbolov (znakov).

Definícia 3.1.2. **Slovo** je *konečná* postupnosť znakov (z nejakej abecedy Σ). Prázdne slovo sa označuje ε .

Definícia 3.1.3. **Jazyk** nad abecedou Σ je ľubovoľná množina slov nad Σ .

Definícia 3.1.4. **Zreťazením slov** $w_1 = a_1 \dots a_n$, $w_2 = b_1 \dots b_m$, $a_1, \dots, a_n, b_1, \dots, b_m \in \Sigma$ dostaneme slovo $w_1 \cdot w_2 = w_1 w_2 = a_1 \dots a_n b_1 \dots b_m$.

Definícia 3.1.5. **Mocnina slova** je definovaná ako $w^0 = \varepsilon$, $w^{n+1} = w^n w$, $\forall n \in \mathbb{N}$.

Poznámka 3.1.1. Abecedu Σ vieme chápať aj ako množinu všetkých jednoznakových slov nad Σ .

Definícia 3.1.6. **Zreťazenie jazykov** L_1 (nad abecedou Σ_1) a L_2 (nad abecedou Σ_2) je jazyk $L_1 \cdot L_2 = L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$ nad abecedou $\Sigma_1 \cup \Sigma_2$.

Definícia 3.1.7. **Mocnina jazyka** L je definovaná ako $L^0 = \{\varepsilon\}$, $L^{n+1} = L^n L$, $\forall n \in \mathbb{N}$.

$$\begin{aligned} L^+ &= \bigcup_{n>0} L^n \\ L^* &= \bigcup_{n\geq 0} L^n \end{aligned}$$

Definícia 3.1.8. Regulárna gramatika je štvorica $G = (N, T, P, \sigma)$, kde N je abeceda neterminálov, T je abeceda terminálov, $P \subseteq_{kon} N \times (T^* \cup T^*N)$ je *konečná* množina prepisovacích pravidiel a $\sigma \in N$ je počiatkový neterminál. Prepisovacie pravidlá teda prepisujú jeden neterminál na ľubovoľné slovo z terminálov za ktorým môže, ale nemusí, nasledovať jeden neterminál.

Definícia 3.1.9. Bezkontextová gramatika je štvorica $G = (N, T, P, \sigma)$, kde N je abeceda neterminálov, T je abeceda terminálov, $P \subseteq_{kon} N \times (N \cup T)^*$ je *konečná* množina prepisovacích pravidiel a $\sigma \in N$ je počiatkový neterminál. Prepisovacie pravidlá teda prepisujú jeden neterminál na ľubovoľné slovo z terminálov a neterminálov.

Poznámka 3.1.2. Pravidlá z P budeme zapisovať v tvare $n \rightarrow w \in P$ namiesto $(n, w) \in P$. Pravidlá s rovnakou ľavou stranou $n \rightarrow w_1, n \rightarrow w_2, \dots, n \rightarrow w_i$ budeme zapisovať zjednodušene ako $n \rightarrow w_1 \mid w_2 \mid \dots \mid w_i$.

Definícia 3.1.10. Krok odvodu v gramatike G je binárna relácia $\Rightarrow_G$ nad $(N \cup T)^*$. Slová x, y sú v relácii $\Rightarrow_G$ práve vtedy, keď $x = w_1nw_2, y = w_1sw_2$ a $n \rightarrow s$ je pravidlo v P .

Definícia 3.1.11. Jazyk generovaný gramatikou G je množina

$$L(G) = \{w \in T^* \mid \sigma \Rightarrow_G^* w\}.$$

Regulárne jazyky, resp. **bezkontextové jazyky** sú jazyky, ktoré sú generované *regulárnou*, resp. *bezkontextovou* gramatikou.

Definícia 3.1.12. Odvodenie slova w (v gramatike G) $\sigma \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w$ je postupnosť krokov odvodení. Pod **ľavým**, resp. **pravým krajným odvodením** rozumieme odvodenie, v ktorom sa v každom kroku odvodenia, v ktorom máme viacero neterminálov, ktoré sa dajú prepisovať, vždy prepíše *najľavejší*, resp. *najpravejší* neterminál.

3.1.2 Syntaktická analýza

Na to, aby sme vedeli preložiť datalog do relačnej algebry, zišlo by sa nám vedieť z čoho sa datalogový program skladá. Aké obsahuje fakty, klauzuly, dotaz, atď. Pomocou týchto informácií bude následne jednoduchšie napísať algoritmus, ktorý nám datalogový program preloží do relačnej algebry. Jedným zo spôsobov je tzv. *syntaktická analýza*. Ukážeme si, ako funguje na jednoduchom príklade, v ktorom budeme prekladať aritmetické výrazy z infixovej notácie do postfixovej notácie.

Zoberme si jednoduchú gramatiku G generujúcu jazyk aritmetických výrazov v infixovej notácii:

$G = (N, T, P, \sigma)$, kde

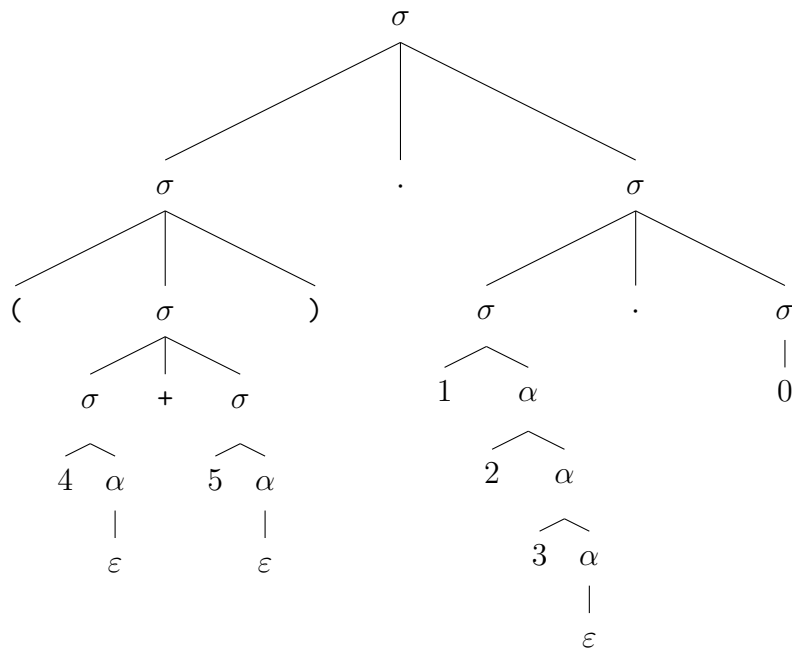
$N = \{\sigma, \alpha\}$,

$T = \{+, \cdot, (,), 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$,

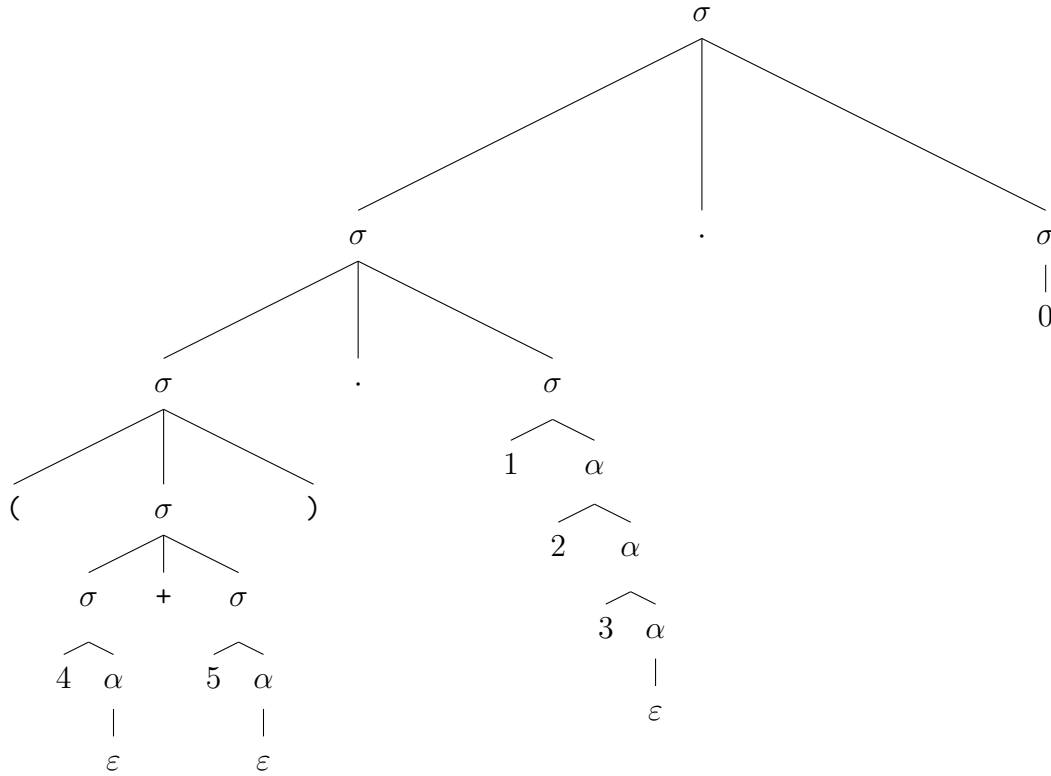
$P = \{\sigma \rightarrow (\sigma) \mid \sigma + \sigma \mid \sigma \cdot \sigma \mid 1\alpha \mid 2\alpha \mid 3\alpha \mid 4\alpha \mid 5\alpha \mid 6\alpha \mid 7\alpha \mid 8\alpha \mid 9\alpha \mid 0,$

$\alpha \rightarrow 0\alpha \mid 1\alpha \mid 2\alpha \mid 3\alpha \mid 4\alpha \mid 5\alpha \mid 6\alpha \mid 7\alpha \mid 8\alpha \mid 9\alpha \mid \varepsilon\}$.

Zoberme si slovo z tohto jazyka, napríklad $(4 + 5) \cdot 123 \cdot 0$, a k nemu jedno prislúchajúce odvodenie (odvodenie zakreslíme pomocou stromu).



Pravé krajné odvodenie



Ľavé krajné odvodenie

Môžeme si všimnúť, že veľkosť stromu odvodenia rýchlo narastá, hlavne kvôli odvodzovaniu čísiel. Preto sa v praxi často používa tzv. *lexer* (*tokenizer*). Lexer rozdelí vstupný reťazec (slovo) na tzv. *tokeny*, ktoré sú zjednodušenou reprezentáciou častí vstupného reťazca. Tieto tokeny sú popísané pomocou regulárnych jazykov (väčšinou pomocou regulárnych výrazov, ktoré sú ekvivalentné s regulárnymi gramatikami [6]). Následne sa odvodenie robí nad tokenami, čím sa zmenší veľkosť stromu odvodenia a zjednoduší sa nám odvodenie. V našom prípade definujeme jeden token reprezentujúci celé čísla

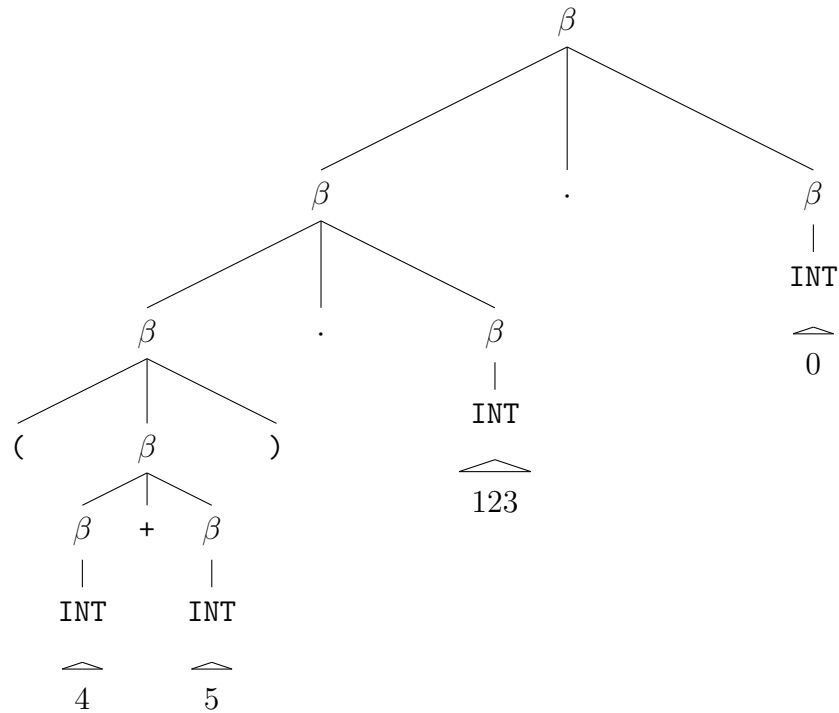
$$\text{INT} = \{0\} \cup \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \cdot \{1, 2, 3, 4, 5, 6, 7, 8, 9, 0\}^* \approx \mathbb{N}_0.$$

Potom sa gramatika G zjednoduší na

$$\begin{aligned} G' &= (N', T', P', \beta), \text{ kde} \\ N' &= \{\beta\}, \\ T' &= \{+, \cdot, (,), \text{INT}\}, \\ P' &= \{\beta \rightarrow (\beta) \mid \beta + \beta \mid \beta \cdot \beta \mid \text{INT}\}. \end{aligned}$$

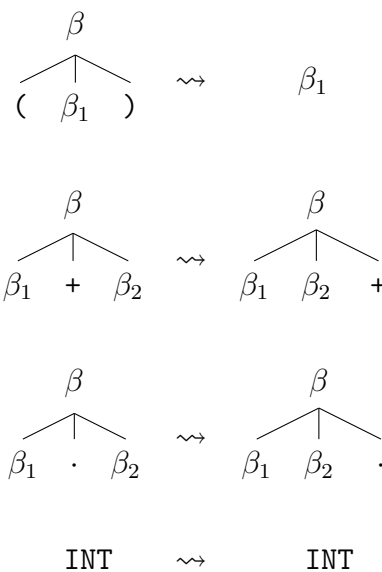
TODO 3.1.1. Lepšie popísať ako funguje lexer.

Ľavé krajné odvodenie slova $(4 + 5) \cdot 123 \cdot 0$ v gramatike G' by vyzeralo nasledovne:



Ľavé krajné odvodenie v gramatike G'

Na takýto strom odvodenia môžeme aplikovať algoritmus, ktorý nám z infixovej notácie vygeneruje postfixovú notáciu. Algoritmus funguje nasledovne: vrcholy stromu (odvodenia) sa transformujú podľa nasledujúcich pravidiel:



Po aplikovaní týchto pravidiel na strom odvodenia dostaneme strom reprezentujúci postfixovú notáciu, z ktorého vieme jednoducho vygenerovať výsledný výraz.

Datalogový program (teória s dotazom) sa skladá z niekoľkých klauzúl (niektoré z nich môžu byť faktami) nasledované dotazom. Do gramatiky (pre ANTLR) to prepíšeme ako

```
program: (clause | definition)* query;
```

kde **program** bude počiatočný neterminál celej gramatiky, **clause**, resp. **definition**, resp. **query** bude neterminál, z ktorého sa odvodí klauzula, resp. definícia, resp. dotaz. Takto definovaná gramatika vyžaduje práve jeden dotaz na konci programu, ale môže obsahovať ľubovoľný počet klauzúl a definícií (vrátane žiadnych).

Neterminál **definition** sa bude prepisovať priamočiaro. Najprv treba zadať meno predikátu, ktorý definujeme a potom napísať n -tice termov, pre ktorý je platný.

```
definition: name BODY_HEAD_SEPARATOR '{' '}' EOL |
            name BODY_HEAD_SEPARATOR '{'
            term_tuple (',' term_tuple)*
            '}' EOL;
```

```
term_tuple: '(' ')' |
            '(' term (',' term)* ')';
```

Neterminál **clause** bude trochu komplikovanejšia, pretože musí obsahovať hlavu a telo klauzuly. V tele klauzuly sa môžu vyskytovať predikáty, negované predikáty, ale aj vstavané predikáty a negované vstavané predikáty.

```
clause: name '(' ')' BODY_HEAD_SEPARATOR
        (predicate | built_in_predicate)
        (',' (predicate | built_in_predicate))* EOL |
        name '(' term (',' term)* ')' BODY_HEAD_SEPARATOR
        (predicate | built_in_predicate)
        (',' (predicate | built_in_predicate))* EOL;
```

```
predicate:      normal_predicate |
                negative_predicate;
negative_predicate: NOT normal_predicate;
```

```
built_in_predicate:      normal_built_in_predicate |
                        negative_built_in_predicate;
negative_built_in_predicate: NOT normal_built_in_predicate;
```

Normálny vstavaný predikát sa skladá z mena a z n -tice termov, rovnako ako hlava klauzuly.

```
normal_predicate: name '(' ')' |
                  name '(' term (',' term)* ')';
```

Vstavaný predikát je podobný normálnemu predikátu, až na to, že ho musíme označiť, že jeho parametre nie sú iba termy, ale aj predikáty a vstavané predikáty, ktoré môžu byť rôzne usporiadané v zoznamoch.

```
normal_built_in_predicate: BUILT_IN name '(' ')' |
                           BUILT_IN name '('
                               parameter (',' parameter)*
                           ')';
```

```
parameter: '[' ']' |
           '[' parameter (',' parameter)* ']' |
           normal_predicate |
           normal_built_in_predicate |
           term;
```

Token `BUILT_IN` by sme mohli vynechať. V takom prípade by ale kompilátor nevedel rozlíšiť vstavané predikáty od normálnych predikátov. To by pri následnom preklade do relačnej algebry znamenalo, že každý predikát, čo nebol definovaný faktom alebo klauzulou, by sa musel považovať za vstavaný predikát. Keďže ale chceme aby kompilátor vedel fungovať samostatne, bez prístupu do *data bázy*, nevieme aké vstavané predikáty existujú. Kvôli tomu by sme nevedeli vyhodíť kompilačnú chybu, keď v klauzule použijeme nedefinovaný predikát.

Keďže povolujeme iba kladné dotazy, tak *query* sa môže prepísať iba na normálny predikát, obyčajný alebo vstavaný.

```
query: QUERY_MARKER (normal_predicate | normal_built_in_predicate) EOL;
```

Použité tokeny sú definované nasledovne, pričom `[0-9]` resp. `[a-z]` znamená ľubovoľný znak z množiny číslíc resp. písmen:


```

QUERY_MARKER: '?-';
BODY_HEAD_SEPARATOR: ':-';
SLC: '%';
MLC_START: '/*';
MLC_END: '*/';
EOL: ' ';
NOT: '\\+';
BUILT_IN: '$';

```

```

UPPER: [A-Z]+;
LOWER: [a-z]+;
NUMBER: [0-9]+;

```

Tieto tokeny nemuseli byť definované, ale na miestach kde sú použité, by sa mohlo použiť ich definíciu. Náš prístup ale dovoľuje používateľovi jednoducho zmeniť tieto tokeny. Napríklad, ak by chcel pre $\leftarrow$ používať alternatívny zápis $<-$, stačí zmeniť definíciu tokenu `BODY_HEAD_SEPARATOR = '<-'`.

Názvy predikátov a funkčných symbolov sa skladajú z malých písmen, čísiel a znaku `_`. Premenné budú rovnaké, ale budú sa skladať z veľkých písmen. Všetky názvy sa musia začať písmenom.

```

name: LOWER (LOWER | NUMBER | '_')*;
function: LOWER (LOWER | NUMBER | '_')*;
variable: UPPER (UPPER | NUMBER | '_')*;

```

Na koniec ešte ANTLR nastavíme, aby preskakoval *whitespace* znaky a podporoval jednoriadkové aj viacriadkové komentáre.

```

WS: [ \t\r\n]+ -> skip;
COMMENT: SLC ~[\n\r]* ([\n\r] | EOF) -> channel(HIDDEN);
MULTILINE_COMMENT: MLC_START (MULTILINE_COMMENT | .)*?
                    (MLC_END | EOF) -> channel(HIDDEN);

```

3.3 Algoritmus prekladu

Po vygenerovaní stromu odvodenia prichádza na rad preklad do relačnej algebry. Preklad sa realizuje v piatich krokoch.

V prvom kroku využijeme dva objekty, ktoré nám automaticky vygeneroval ANTLR. Najprv vytvoríme objekt `datalogLexer`, ktorý potom premeníme na

`CommonTokenStream`. Tento druhý objekt je v podstate zoznam tokenov, ktoré sa vyskytujú v zdrojovom kóde. Formálne by to boli znaky (resp. písmená) slova, z ktorých hľadáme strom odvodenia.

```
CharStream program;
datalogLexer lexer = new datalogLexer(program);
CommonTokenStream tokens = new CommonTokenStream(lexer);
```

V druhom kroku vytvoríme objekt `datalogParser` (ktorý tiež vygeneroval ANTLR), ktorý nám umožní pracovať s gramatikou, ktorú sme definovali v súbore `datalog.g4`. Tento objekt nám umožní vytvoriť strom odvodenia z tokenov, ktoré sme získali v prvom kroku.

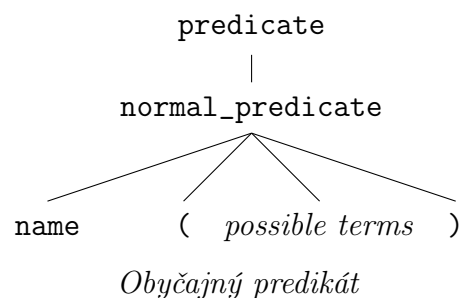
```
datalogParser parser = new datalogParser(tokens);
ParseTree tree = parser.program();
```

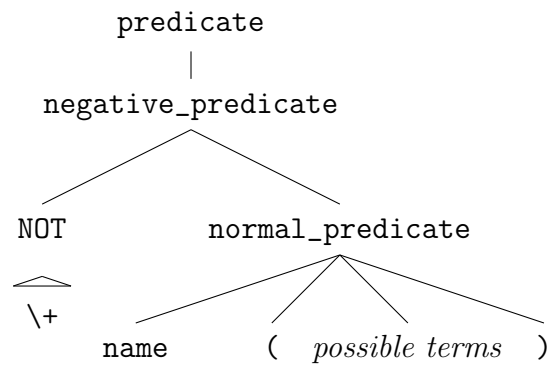
3.3.1 Generácia AST

Tretí krok je zavolanie objektu `DatalogASTBuilder`, ktorý nám transformuje strom odvodenia na abstraktný syntaktický strom (AST). Tento strom je jednoduchší a pohodlnejší na ďalšie spracovanie. Transformácia sa realizuje pomocou tzv. *visitor patternu*. Vstupom je objekt typu `DatalogProgram`, ktorý obsahuje všetky definície, klauzuly a dotaz. Triedy, ktoré sa týkajú AST, sú definované v priečinkoch `datalog` a `common` (spoločné objekty pre `datalog` aj relačnú algebru, termy a parametre).

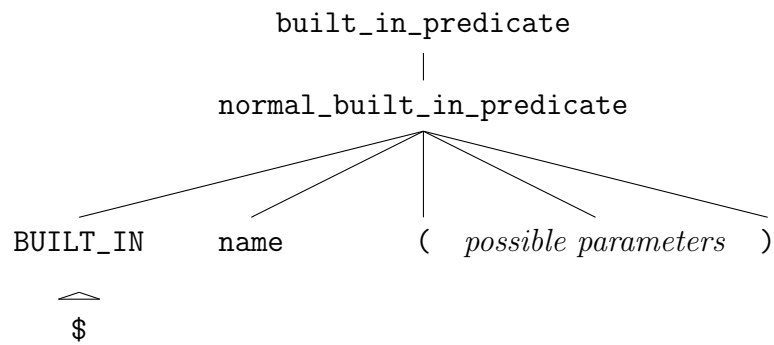
```
DatalogASTBuilder astBuilder = new DatalogASTBuilder();
DatalogProgram ast = (DatalogProgram) astBuilder.visit(tree);
```

Príkladom, aká transformácia sa deje, je transformácia na objekt typu `Atom` a `Predicate`. Všetky nasledujúce podstromy odvodenia sa transformujú na rovnaké objekty s rovnakým rozhraním.

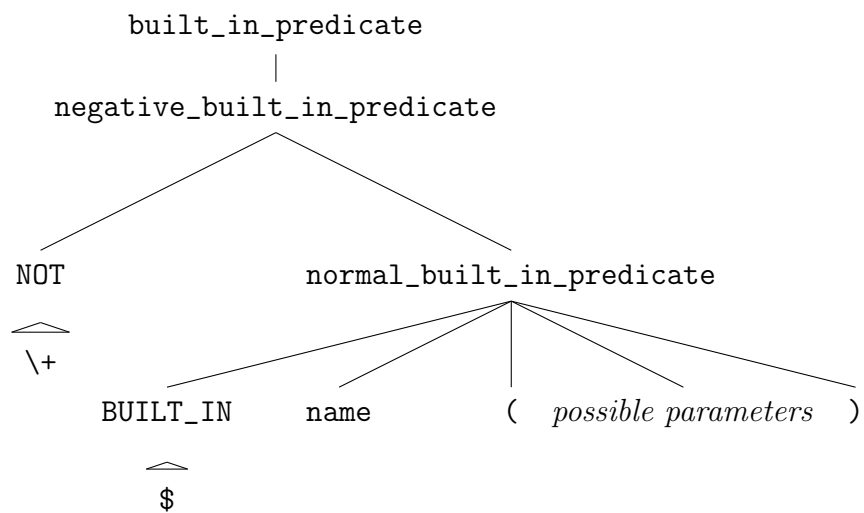




Negovaný obyčajný predikát

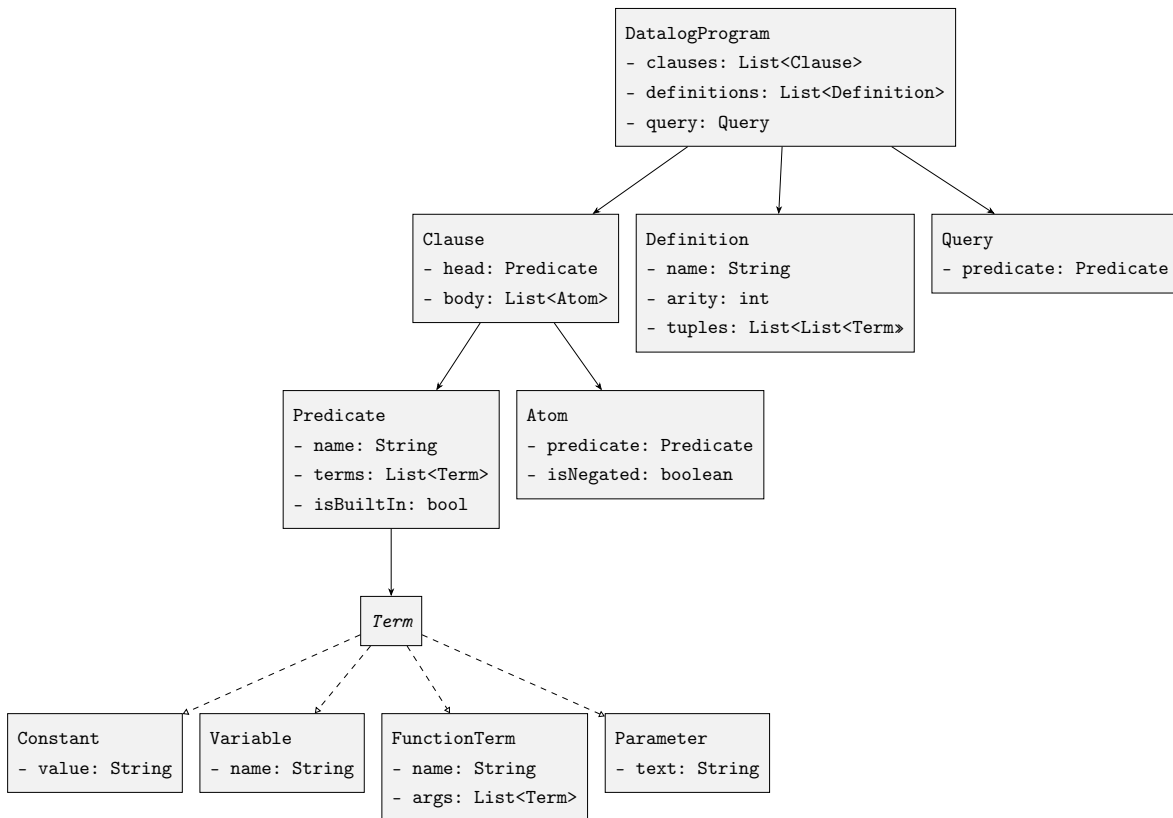


Vstavaný predikát



Negovaný vstavaný predikát

Celá štruktúra a hierarchia tried, ktoré reprezentujú AST, sú podrobnejšie popísané v diagrame.



Hierarchia tried AST

V tejto fáze sa tiež realizuje kontrola, či sú klauzuly bezpečné a či všetky `term_tuple` v definícii majú rovnakú aritu, keďže tieto vlastnosti programu nie je možné vyjadriť pomocou bezkontextovej gramatiky. Taktiež sa kontroluje, či termy v `term_tuple` neobsahujú premenné. Táto vlastnosť by sa dala vyjadriť pomocou bezkontextovej gramatiky, ale pre jednoduchosť implementácie sme sa rozhodli proti tomu.

3.3.2 Preklad do relačnej algebry

Posledné 2 kroky sa týkajú samotného prekladu do relačnej algebry a vykonáva ich trieda `DatalogCompiler`.

```

DatalogCompiler compiler = new DatalogCompiler();
RAExpr compiled = compiler.compile(ast);

```

V štvrtom kroku vygenerujeme pre každý predikát v programe objekt typu `RAExpr`, ktorý reprezentuje reláciu k nemu prislúchajúcu. Ak je nejaký predikát závislý na inom predikáte, zatiaľ ho nepreložíme, ale použijeme referenciu (*placeholder*) na tento predikát (objekt typu `RelationRef`). Prekladať nebudeme priamo do relačnej algebry, ale do Java objektov reprezentujúcich relačnú algebru. To nám umožní ľahšie manipulovať s výsledným výrazom a taktiež jednoduchšie meniť zápis výsledného výrazu. Napríklad

ak by sme chceli zmeniť poradie parametrov v operátoroch, alebo zmeniť odsadenie a zátvorky.

Prvým problémom, s ktorým sa v tomto kroku stretávame, je ako preložiť definície. Definície sú v podstate relácie, ktoré obsahujú pevne dané množiny n -tíc. Keďže nemáme žiaden operátor, ktorý by toto priamo podporoval, musíme „zneužiť“ operátor **JOIN** a vstavané relácie.

Definícia 3.3.1. Vstavané relácie $\$true$ a $\$false$ sú definované nasledovne:

$$\begin{aligned}\$true &= \{()\} \\ \$false &= \emptyset\end{aligned}$$

Zoberme si výraz **JOIN**($[\$true]$, $[\square]$, $[n1(), a()]$) (normálne by sme zapísali konštanty ako 1 a a , ale naša aktuálna implementácia gramatiky vyžaduje tento zápis). Nech M je množina z definície **JOIN**. Potom $M = \{()\}$, keďže iba pre $()$ platí $() \in \$true$. Zápis je trochu neintuitívny, ale zhoduje sa s definíciou, keďže neexistuje žiadny term $T_{i,j}$ ani premenná P_k . Potom výsledná relácia R je rovná $\{(n1(), a())\}$.

Týmto spôsobom vieme vyrobiť ľubovoľnú reláciu s jednou n -ticou. Pre viacnásobné n -tice môžeme použiť viacnásobné **JOIN** obalené v **UNION**.

Program (resp. iba časť programu) `color :- {(red()), (blue())}`. sa preloží ako

```
UNION([
  JOIN([$true], [\square], [red()]),
  JOIN([$true], [\square], [blue()])
])
```

Príklad 3.1

Keďže jeden predikát môže byť definovaný viackrát, musíme prejsť všetky definície, pozbierať všetky n -tice a spojiť ich do jednej relácie (bez duplikátov).

Pôvodne sme chceli mať 5 operátorov. Piaty operátor by bol **DEF**, ktorý by nám umožňoval definovať reláciu pomocou množiny n -tíc. Preto sa používa v programe kompilátora trieda **RADef**. Keďže sme sa ale nakoniec rozhodli uprednostniť menšiu množinu operátorov, **DEF** sa transformuje na **UNION JOIN**-ov, ako sme ukázali v príklade vyššie. Preto sa v tejto fáze realizuje aj táto transformácia.

Druhým problémom je preklad klauzúl. V datalogu je klauzula v podstate konjunkcia atómov a keďže konjunkcia je asociatívna a komutatívna, môžeme atómy v klauzule zoradiť ľubovoľne. Preto môžeme pred prekladom klauzúl zoradiť atómy tak, že najprv budú kladné atómy a až potom negované atómy.

Z kladných atómov vieme pomocou **JOIN** vyrobiť reláciu, ktorá obsahuje všetky n-tice, ktoré môžeme doplniť, aby boli všetky pravdivé. Túto výslednú reláciu potom použijeme ako prvú reláciu v **ANTIJOIN** operátore, pomocou ktorého odfiltrujeme všetky n-tice, ktoré by spĺňali negované atómy. Výstupom celej klauzuly sú ale iba tie n-tice, ktoré spĺňajú hlavu klauzuly. To zohľadníme vo výstupnom vektore v **ANTIJOIN**-e (respektíve v **JOIN**-e, ak nemáme negatívne atómy).

Časť datalogového programu

```
dvojica(f(A), D) :- $hrana(A, B),
                    $hrana(f(B), kon()),
                    $hrana(kon(), D),
                    \+ $hrana(A, kon()),
                    \+ $hrana(f(B), D).
```

sa preloží nasledovne. Najprv vytvoríme reláciu z kladných atómov pomocou **JOIN**-u.

```
JOIN(
    [$hrana, $hrana, $hrana],
    [[A, B], [f(B), kon()], [kon(), D]],
    [A, B, D]
)
```

Táto relácia obsahuje všetky n-tice, ktoré spĺňajú kladné atómy. Potom použijeme **ANTIJOIN** na vytvorenie výslednej relácie *dvojica*.

```
ANTIJOIN(
    [JOIN(
        [$hrana, $hrana, $hrana],
        [[A, B], [f(B), kon()], [kon(), D]],
        [A, B, D]
    ),
    $hrana, $hrana],
    [[A, B, D], [A, kon()], [f(B), D]],
    [f(A), D]
)
```

Príklad 3.2

V prípade, že klauzula neobsahuje žiadne negované atómy, použijeme iba **JOIN** a výstupný vektor bude zodpovedať hlave klauzuly. V prípade, že klauzula obsahuje iba

jeden pozitívny atóm, **JOIN** nie je potrebné vyrábať, ale môžeme použiť tento atóm priamo v **ANTIJOIN**-e.

Zaujímavý prípad je klauzula, ktorá obsahuje iba negované atómy. Keďže musí byť bezpečná, nemôže obsahovať žiadne premenné, ale iba konštanty. V takomto prípade má klauzula tvar „*n-tica 1 patrí do relácie A, ak n-tica 2 nepatrí do relácie B, n-tica 3 nepatrí do relácie C, ...*“. Keďže ale **ANTIJOIN** potrebuje jednu reláciu v pozitívnom kontexte, musíme do všetkých takýchto klauzúl pridať nejaký pozitívny atóm, napríklad `$true()`.

Klauzula `test(n1()) :- \+ $a(n2()), \+ $b(n3())` sa preloží ako:

`ANTIJOIN([$true, $a, $b], [], [n2()], [n3()], [n1()]`

Príklad 3.3

V prípade, že jeden predikát je definovaný viackrát, musíme všetky klauzuly, ktoré mu prislúchajú, spojiť do jednej relácie pomocou **UNION**-u.

Program

`p(X) :- $a(X).`

`p(Y) :- $b(Y).`

sa preloží ako:

`UNION([
 JOIN([$a], [[X]], [X]),
 JOIN([$b], [[Y]], [Y])
])`

Príklad 3.4

Výsledkom štvrtého kroku je teda mapa `Map<String, RAExpr>`, ktorá priradzuje každému predikátu (resp. relácii) výraz v relačnej algebre, ktorý ho reprezentuje. Ak sa v niektorej relácii vyskytuje referencia na iný predikát, táto referencia je zatiaľ nevyplnená (je to objekt typu `RelationRef`).

Poznámka 3.3.1. Teoreticky nezáleží od poradia atómov v klauzule, ale pri konkrétnych vstavaných predikátoch môže záležať. Zoberme si napríklad predikát `$plus(X, Y, Z)`, ktorý je pravdivý vtedy a len vtedy, keď $X + Y = Z$. Jeho implementácia v databázovom systéme môže byť taká, že ak sú zadané (sú konštanty) X a Y , vráti Z (Z je premenná) alebo ak sú zadané X , Y a Z , vráti pravdivostnú hodnotu a to je všetko. Aktuálna implementácia napr. **JOIN**-u v našom databázovom systéme je ale nasledovná. Zoberie sa prvá relácia a jej vstupný vektor. Vyberie sa z nej nejaká n -tica,

aby spĺňala vstupný vektor. Týmto sa môžu konkretizovať niektoré premenné. Potom sa zoberie druhá relácia a jej vstupný vektor (s už konkretizovanými premennými) a spravý sa to isté. Preto v našom prípade záleží od poradia atómov. Ako príklad si zoberme program

```
p(Z) :- $cisloDo10(X), $cisloDo10(Y), $plus(X, Y, Z).
```

Prislušný výraz v relačnej algebre je

```
JOIN(
  [$cisloDo10, $cisloDo10, $plus],
  [[X], [Y], [X, Y, Z]],
  [Z]
)
```

Sémanticky identický program

```
p(Z) :- $plus(X, Y, Z), $cisloDo10(X), $cisloDo10(Y).
```

sa ale preloží ako

```
JOIN(
  [$plus, $cisloDo10, $cisloDo10],
  [[X, Y, Z], [X], [Y]],
  [Z]
)
```

a tento výraz nemusí byť správne vyhodnotený, keďže *plus* nemusí vedieť pracovať so stroma premennými.

Preto si používateľ musí dávať pozor na to, v akom poradí píše atómy do klauzúl, keďže to môže ovplyvniť výsledok. Naš kompilátor zoberie naprv kladné atómy v poradí a následne negované atómy v poradí v akom boli zapísané v programe.

V piatom kroku tieto referencie nahradíme výrazom, ktorý im prislúcha. Keďže ale môže nastať situácia, že niektorý predikát je závislý sám na sebe (priamo alebo nepriamo), musíme prípadne predikáty *zabaliť* do rekurzívneho operátora **REC**. To spravíme tak, že naprv skonštruujeme graf závislosti predikátov, v ktorom bude orientovaná hrana z predikátu *A* do predikátu *B*, ak *A* (priamo) závisí na *B*. Potom pre každý predikát skontrolujeme, či existuje cesta z neho do seba samého. Ak áno, tento predikát je rekurzívny a musíme ho zabaliť do **REC**.

Výsledný výraz v relačnej algebre začneme stavať od dotazu. Prechádzame jeho výraz do hĺby a pamätáme si, cez aké **REC** operátory sme prešli. Keď narazíme na

RelationRef (pre ktorý sme nevideli operátor **REC**), nahradíme ho výrazom, ktorý mu prislúcha. Takto postupujeme, až kým nenahradíme všetky **RelationRef** výrazmi, ktoré im prislúchajú. Výsledkom ale nemá byť celá relácia, ale iba tie záznamy, ktoré spĺňajú hlavu dotazu. Preto ešte celý výsledný výraz zabalíme do **JOIN**-u, ktorého výstupný vektor bude zodpovedať hlave dotazu.

Ukážeme si preklad programu spolu z vysvetlením, ktoré časti relačnej algebry zodpovedajú ktorým častiam programu.

```

1  node :- {(a()), (b()), (c())}.
2  blocked :- {(c())}.
3
4  reachable(X) :- node(X).
5  reachable(X) :- reachable(X).
6
7  selected(X) :- reachable(X), $eq(X, b()).
8  free(X) :- reachable(X), \+ blocked(X).
9
10 result(X) :- free(X), selected(X).
11
12 ?- result(Z).
```

Znak % označuje jednoriadkový komentár.

```

JOIN([                                     % dotaz - 12
  JOIN([                                   % result - 10
    ANTIJOIN([                             % free - 8
      REC(reachable,                       % lebo riadok 5 je rekurzívny
        UNION([
          JOIN([                             % reachable - 4
            UNION([                         % node - 1
              JOIN([$true], [[]], [a()]),
              JOIN([$true], [[]], [b()]),
              JOIN([$true], [[]], [c()])
            ])
          ], [[X]], [X]),
          JOIN([reachable], [[X]], [X])    % reachable - 5
        ])
      ),
      JOIN([$true], [[]], [c()])          % blocked - 2
    ], [[X], [X]], [X]),
```

```

JOIN([                                     % selected - 7
  REC(reachable,                          % znovu reachable
    UNION([
      JOIN([
        UNION([
          JOIN([$true], [[]], [a()]),
          JOIN([$true], [[]], [b()]),
          JOIN([$true], [[]], [c()])
        ])
      ], [[X]], [X]),
    JOIN([reachable], [[X]], [X])])
  ),
  $eq
], [[X], [X, b()]], [X])
], [[X], [X]], [X])
], [[Z]], [Z])

```

*Príklad 3.5***Rekapitulácia**

Preklad do relačnej algebry sa realizuje v piatich krokoch. Heslovite sú to:

1. Vytvorenie lexeru a token streamu
2. Vytvorenie parseru a stromu odvodenia
3. Generácia AST z odvodenia
4. Preklad predikátov do relačnej algebry s nevyplnenými referenciami
 - (a) Preklad definícií pomocou **JOIN** a **UNION**
 - (b) Preklad klauzúl pomocou **JOIN** a **ANTIJOIN**
 - (c) Spojenie klauzúl do jednej relácie pomocou **UNION**
5. Postavenie výsledného výrazu v relačnej algebre
 - (a) Postavenie grafu závislosti predikátov
 - (b) Zabalenie rekurzívnych predikátov do **REC**
 - (c) Nahradenie referencií výrazmi, ktoré im prislúchajú
 - (d) Zabalenie výsledného výrazu do **JOIN**-u reprezentujúceho dotaz

Záver

Literatúra

- [1] ANTLR Project. Antlr 4 documentation. <https://github.com/antlr/antlr4/blob/master/doc/index.md>, 2026. [Online; navštívené 21. apríla 2026].
- [2] Diana Biriukova. Compiler of relational algebra expressions, 2025.
- [3] Tomáš Magát. Todo, 2026.
- [4] Tomáš Plachetka and Ján Štunc. Semantics of queries. Prednáškové slajdy ku kurzu Databases, Fakulta matematiky, fyziky a informatiky Univerzity Komenského v Bratislave, February 2026. Summer 2025–2026. Dostupné na: <http://www.dcs.fmph.uniba.sk/~plachetk/TEACHING/RLDB2026/index.html>.
- [5] Branislav Rován and Michal Forišek. Definície. Vo: Formálne jazyky a automaty, 2013. s. 7. – 12. FMFI UK, Bratislava.
- [6] Branislav Rován and Michal Forišek. Regulárne výrazy. Vo: Formálne jazyky a automaty, 2013. s. 28. FMFI UK, Bratislava.
- [7] Branislav Rován and Michal Forišek. Syntaktická analýza. Vo: Formálne jazyky a automaty, 2013. s. 102. – 114. FMFI UK, Bratislava.
- [8] Soufflé Language Team. Soufflé: Program translation. <https://souffle-lang.github.io/translate>, 2024. [Online; navštívené 7. januára 2026].
- [9] Allen Van Gelder, Kenneth A. Ross, and John S. Schlipf. The well-founded semantics for general logic programs. *Journal of the ACM*, 38(3):620–650, July 1991.